AIN SHAMS UNIVERSITY

Faculty of Engineering
Computer Engineering and Artificial Intelligence Engineering

MicroGrad

Assignment 1: Scalar Automatic Differentiation Engine

Documentation Report

Course: CSE485–CSE488: Deep Learning

Student: Ali Ashraf Hamdy

Student ID: 23P0233

Assignment: MicroGrad (Scalar Autograd Engine)

October 2026

Contents

1 Introduction and Objectives **2**

2 Computational Graph and Value Representation **2**
2.1 Scalar nodes . . . 2
2.2 Addition . . . 2
2.3 Multiplication . . . 2
2.4 Power . . . 3
2.5 Exponential . . . 3
2.6 Hyperbolic tangent . . . 3

3 Backpropagation and the Chain Rule **3**
3.1 Chain rule . . . 3
3.2 Why reverse topological order is required . . . 4
3.3 The diamond-graph case . . . 4

4 Neural Network API **4**
4.1 Neuron . . . 4
4.2 Layer . . . 5
4.3 Multi-layer perceptron . . . 5

5 Training Objective and Parameter Updates **5**

6 Demonstration Experiment **5**
6.1 Results to record after executing the notebook . . . 6

7 Design Justifications **6**

8 Conceptual Discussion **6**
8.1 Why does the engine store a computation graph? . . . 6
8.2 Why is the output gradient initialized to one? . . . 6
8.3 Why use a visited set during topological sorting? . . . 7
8.4 Why are gradients accumulated? . . . 7
8.5 What is the role of the activation function? . . . 7

9 Conclusion **7**

1 Introduction and Objectives

The objective of this assignment is to build a small scalar automatic differentiation engine inspired by MicroGrad. Rather than relying on a deep-learning framework for differentiation, the engine represents a computation as a directed acyclic graph (DAG), stores local derivative information, and applies the chain rule during a backward pass.

The assignment has two implementation components:

- `engine.py`: scalar values, operation overloading, local derivatives, activation functions, and graph-based backpropagation.
- `nn.py`: the `Neuron`, `Layer`, and `MLP` abstractions, including parameter initialization and forward computation.

The accompanying demonstration notebook is designed to load a small dataset, train an MLP using the custom engine, and visualize the resulting loss and decision boundary.

2 Computational Graph and Value Representation

2.1 Scalar nodes

Each `Value` represents a scalar in the computation graph. Conceptually, a node stores:

- its numerical value, denoted by x;
- its accumulated gradient, denoted by $\frac{\partial L}{\partial x}$;
- references to its immediate predecessor nodes;
- a local backward function that propagates the node's gradient to its predecessors.

An operation creates a new node whose predecessors are the input nodes. This preserves the history of the computation and makes it possible to differentiate a final scalar output with respect to earlier inputs and parameters.

2.2 Addition

For

$$z = a + b,$$

the local derivatives are

$$\frac{\partial z}{\partial a} = 1, \qquad \frac{\partial z}{\partial b} = 1.$$

If the upstream gradient is $\frac{\partial L}{\partial z}$, the chain rule gives

$$\frac{\partial L}{\partial a} \mathrel{+}= \frac{\partial L}{\partial z}, \qquad \frac{\partial L}{\partial b} \mathrel{+}= \frac{\partial L}{\partial z}.$$

The accumulation operator is important because one node may contribute to an output through multiple paths.

2.3 Multiplication

For

$$z = ab,$$

the local derivatives are

$$\frac{\partial z}{\partial a} = b, \qquad \frac{\partial z}{\partial b} = a.$$

Therefore,

$$\frac{\partial L}{\partial a} \mathrel{+}= b\frac{\partial L}{\partial z}, \qquad \frac{\partial L}{\partial b} \mathrel{+}= a\frac{\partial L}{\partial z}.$$

Each input receives the upstream gradient multiplied by the other input's forward value.

2.4 Power

For a constant exponent n,

$$z = a^n, \qquad \frac{\partial z}{\partial a} = na^{n-1}.$$

Thus,

$$\frac{\partial L}{\partial a} \mathrel{+}= na^{n-1}\frac{\partial L}{\partial z}.$$

This is the scalar power rule used when the exponent is a constant. A general implementation should make clear which exponent cases it supports.

2.5 Exponential

For

$$z = e^a,$$

the derivative is

$$\frac{\partial z}{\partial a} = e^a = z.$$

Consequently,

$$\frac{\partial L}{\partial a} \mathrel{+}= z\frac{\partial L}{\partial z}.$$

2.6 Hyperbolic tangent

For

$$z = \tanh(a),$$

the derivative is

$$\frac{\partial z}{\partial a} = 1 - \tanh^2(a) = 1 - z^2.$$

The propagated gradient is therefore

$$\frac{\partial L}{\partial a} \mathrel{+}= (1 - z^2)\frac{\partial L}{\partial z}.$$

The output value z can be reused in the backward calculation.

3 Backpropagation and the Chain Rule

3.1 Chain rule

Suppose an intermediate value u affects a scalar loss L through v. The chain rule states

$$\frac{\partial L}{\partial u} = \frac{\partial L}{\partial v}\frac{\partial v}{\partial u}.$$

For a computational graph, there may be several downstream paths from u to L. The total derivative is the sum of the contributions from all such paths. This is why gradients must be accumulated rather than overwritten.

3.2 Why reverse topological order is required

A node's gradient is the sum of contributions from all nodes that depend on it. Therefore, all downstream contributions must arrive before the node propagates its gradient to its own predecessors. A topological ordering places each predecessor before the node that depends on it; reversing that order processes downstream nodes before upstream nodes.

A depth-first-search post-order traversal can construct the required ordering:

1. Start from the output node.
2. Visit each predecessor recursively if it has not already been visited.
3. Append the current node only after all its predecessors have been visited.
4. Reverse the resulting post-order list for the backward pass.

The output gradient is seeded with 1, because

$$\frac{\partial L}{\partial L} = 1.$$

Then each node's local backward function distributes its gradient to its predecessors.

3.3 The diamond-graph case

Consider

$$u = a + b, \qquad v = a \cdot b, \qquad L = u + v.$$

The value a contributes to L through both u and v. Its derivative is

$$\frac{\partial L}{\partial a} = \frac{\partial u}{\partial a} + \frac{\partial v}{\partial a} = 1 + b.$$

Similarly,

$$\frac{\partial L}{\partial b} = 1 + a.$$

This example demonstrates why a node may be reached through multiple paths and why its gradient contributions must be added. A visited set in the topological traversal prevents redundant traversal of shared nodes; it does not replace gradient accumulation.

4 Neural Network API

4.1 Neuron

A fully connected neuron computes a weighted sum plus a bias:

$$z = \sum_{i=1}^{d} w_i x_i + b.$$

With an activation function ϕ, its output is

$$\hat{y} = \phi(z).$$

The weights and bias are trainable scalar parameters represented as graph nodes. The input values are supplied by the data.

4.2 Layer

A layer contains multiple neurons that receive the same input vector. If a layer has m neurons, it produces m outputs:

$$\mathbf{h} = \left[\phi\left(\sum_i w_{1i} x_i + b_1 \right), \ldots, \phi\left(\sum_i w_{mi} x_i + b_m \right) \right].$$

The layer abstraction groups neurons and provides a convenient forward operation.

4.3 Multi-layer perceptron

An MLP composes layers sequentially. For layer ℓ,

$$\mathbf{h}^{(\ell)} = \phi^{(\ell)}\left(W^{(\ell)} \mathbf{h}^{(\ell-1)} + \mathbf{b}^{(\ell)} \right),$$

where $\mathbf{h}^{(0)} = \mathbf{x}$. The network's parameters are all weights and biases across its layers.

Random initialization breaks symmetry between neurons. If all neurons in a layer began with identical parameters, they would receive identical gradients and could learn identical features. Random initial values help neurons develop different parameter updates.

5 Training Objective and Parameter Updates

The demonstration notebook uses a binary two-dimensional dataset with target labels $y \in \{-1, +1\}$. For a scalar network prediction $\hat{y}_i$, the mean squared error over N examples is

$$L = \frac{1}{N} \sum_{i=1}^{N} (\hat{y}_i - y_i)^2.$$

The backward pass computes the gradient of this scalar loss with respect to every trainable parameter θ. Gradient descent then updates each parameter:

$$\theta \leftarrow \theta - \eta \frac{\partial L}{\partial \theta},$$

where η is the learning rate.

Gradients must be reset before computing a new training iteration; otherwise, gradients from previous iterations would be added to the new gradients. The forward pass constructs the loss graph, `backward()` computes derivatives, and the update step changes the parameter data.

6 Demonstration Experiment

The accompanying notebook is designed around a reproducible concentric-circles classification task:

- Two input features describe each point in the plane.
- The inner circle is assigned label -1 and the outer circle label $+1$.
- The generated samples are written to a CSV file and loaded back.
- The data is split into training and test subsets using a fixed random seed.
- An MLP with layer widths $[16, 16, 1]$ is trained using the custom MicroGrad implementation.
- Training loss and a two-dimensional decision-boundary visualization are produced.

The purpose of this experiment is to demonstrate that the scalar engine can be composed into a neural network and used to optimize model parameters on a non-linear classification problem.

6.1 Results to record after executing the notebook

The assignment brief requires the notebook to show training and final visualizations. The numerical results below must be copied from the successfully executed notebook; they are intentionally not fabricated in this report.

Metric	Measured result
Final training MSE	*Record from notebook*
Test MSE	*Record from notebook*
Test classification accuracy	*Record from notebook*

Insert or retain the executed training-loss plot and decision-boundary plot in the final notebook. If the measured results differ from expectations, inspect the implementation and training configuration rather than changing the reported values.

7 Design Justifications

- **Scalar values:** scalar nodes keep the engine small and make the chain rule and graph structure explicit.
- **Operator overloading:** ordinary arithmetic expressions can build the graph without requiring users to manually construct operation nodes.
- **Local backward functions:** each operation is responsible for its own derivative, keeping differentiation modular.
- **Gradient accumulation:** adding contributions supports graphs in which a value is reused in multiple expressions.
- **Topological sorting:** reverse topological order ensures downstream gradients are accumulated before upstream propagation.
- **Neuron/layer/MLP abstractions:** these separate individual computation units, groups of neurons, and full network composition.
- **Small synthetic dataset:** the task is fast to generate and visualize, while its circular boundary requires a non-linear model.

8 Conceptual Discussion

8.1 Why does the engine store a computation graph?

The graph records how an output depends on its inputs. Once the forward computation has been performed, the graph provides the dependency structure needed to apply local derivatives in reverse order and calculate gradients with respect to earlier values and parameters.

8.2 Why is the output gradient initialized to one?

The backward pass starts from a scalar output L. Its derivative with respect to itself is one, so the initial gradient seed is $\partial L/\partial L = 1$. All other gradients follow by repeated application of the chain rule.

8.3 Why use a visited set during topological sorting?

A computation graph is a DAG and can contain shared predecessor nodes. The visited set ensures that each node is added to the topological ordering once, avoiding repeated traversal of shared subgraphs. Gradient contributions from separate paths are still accumulated during backpropagation.

8.4 Why are gradients accumulated?

A parameter or intermediate value can affect the loss through more than one path. The total derivative is the sum of the path-wise derivatives. Overwriting a gradient would discard contributions and produce an incorrect result.

8.5 What is the role of the activation function?

Without non-linear activation functions, a composition of affine layers is still an affine function. Non-linear activations allow an MLP to represent non-linear decision boundaries, such as the circular boundary in the demonstration task.

9 Conclusion

This assignment connects scalar differentiation, graph traversal, the chain rule, and neural network training. The engine's core responsibility is to record scalar operations and propagate gradients correctly. The neural-network API builds trainable models from these scalar operations. The accompanying notebook provides an end-to-end experiment that exercises the implementation on a small classification task.

Submission Note

The assignment PDF specifies that the LMS ZIP must include the cleaned codebase, a reproducible Jupyter notebook, and a LaTeX-compiled PDF report. It does not enumerate additional conceptual questions or prescribe a particular dataset or network architecture. This report therefore addresses the stated engine and neural-network concepts and documents the experiment planned in the accompanying notebook.